Chat Completions

Explore how to use OpenAI's chat completions endpoint for tasks like generating, classifying, and transforming text. Understand request and response parameters, and how prompt design impacts output to create effective NLP solutions in Python.

The chat completions endpoint

The **chat completions** endpoint can be used to perform many tasks on a text, including classification, generation, transformation, completion of incomplete text, factual responses, and others. The endpoint takes an input message from the user and the role assigned to it and returns a JSON object.

Chat completions

Ask

Completions API call

In the code segment below, we use the `openai` library in Python. The following function can be used to call the completions endpoint:

```python
from openai import OpenAI
client = OpenAI(api_key="{{YOUR_SECTEY_KEY}}")

response = client.chat.completions.create(
  model="model_id",
  messages=[
    {"role": "system", "content": "..."},
    {"role": "user", "content": "..."}
  ],
)
```

Understanding the chat completions endpoint

Let's look at the chat completions endpoint in more detail, reviewing the request parameters and the response parameters.

Request parameters

Let's see some essential request parameters for this endpoint in the table below:

Fields	Format	Type	Description
`messages`	Array	Required	This is a list of messages comprising the con
`model`	String	Required	This is the ID of the model that the chat com
`temperature`	Float	Optional	Which sampling temperature shoul `2` ? The default temperature is set a as `0.8` , will increase randomness in the outp like `0.2` , will enhance focus and determinisr

Mini Map
Ask

`top_p`	Float	Optional	**Nucleus sampling** is an alternative to tempe the model evaluates the outcomes of tokens mass. So, 0.1 indicates that only the top 10% will be evaluated. Default value: `1`
`response_format`	object	Optional	This is an object that specifies the format of t is compatible with the newer models. Setting to `{ "type" : "json_object" }` wil object.
`n`	Integer	Optional	It indicates the number of chat completions t Default value: `1`
`logprobs`	Integer	Optional	This value decides whether to return the log tokens. This option is not available on the `gp` model. Default value: `false`
`stop`	String/array	Optional	This allows you to provide four sequences wl generating further tokens. Default value: `null`
`presence_penalty`	Float	Optional	A value range between -2.0 to 2.0. The positi new tokens if they exist in the text, increasing generating new things. Default value: `0`
`frequency_penalty`	Float	Optional	A value range between -2.0 to 2.0. The positi new tokens concerning their existing frequen the possibility of repeating the same words. Default value: `0`

> **Note**: You can learn more about the `engine` argument in this lesson.

Ask

Response fields

The response is a JSON object. Some essential attributes are given below:

Fields	Format	Description
`id`	String	This is a unique ID for the chat completions.
`choices`	Array	It is an array of objects. Every object contains val information about the response. The size of the a equal to the n parameter that we provided in the parameters.
`model`	String	This is the Unix timestamp (in seconds) of when t completions was created.
`usage`	Object	These usage statistics specify the `prompt_token` `completion_tokens` , and `total_tokens` for the completion requests.

From these response parameters, the parameter `choices` contains the output response from the the API. Let's look at the `choices` array in more detail.

Fields	Format	Descri
`choices[i].finish_reason`	String	This provides the reas stopped generating to reasons sent: `stop` : This is a natura `length` : The number in the request reached `content_filter` : Cor a flag genera `tool_calls`
`choices[i].index`	Integer	This gives the index of of choices.

Ask

`choices[i].message`	Object	This is the chat compl message generated b
`choices[i].logprobs`	Object	This is the log probabi choice.

The input to this endpoint is an array that contains the request text message and role for the system. The output is also an array that contains a role and the text message response.

Completions: input and output model

Prompt design

The chat completions endpoint is an array-in and array-out endpoint. We can instruct it on what to do by providing the relevant texts in the array. A well-written prompt will result in good output.

Ask

Prompt design

Basics of prompt design

OpenAI can perform many simple and complex tasks in text analysis. To get the most out of it, we have to be very specific about our prompt. We must take particular care about the following points:

- Proofread the prompt for spelling and grammatical mistakes because that can affect the output.
- Check for setting parameters such as `temperature` and `top_p`.
- Ensure clarity about the intended generation of the text.
- Provide one or more examples. (We'll see this prompt design in the next lessons.)

Let's look at a few examples by executing various prompts.

Using the chat completions endpoint

Let's use the chat completions endpoint to write something about "artificial intelligence." In the code widget below, we use the GPT-3.5-Turbo model for the chat completions task. The `temperature` value used is `0.9`. Make sure you have added your `SECRET_KEY`. Press the "Run" button to see the response for our `prompt`.

Ask

The following prompt exhibits the generation use case by generating some text corresponding to the provided prompt:

```
Write a tagline about artificial intelligence.
```

We may get the following output:

```
Unlocking the potential of artificial intelligence, one innovation at a
```

> **Note:** The output can be different for each run.

Let's try out the prompt provided above.

Please provide values for the following:

SECRET_KEY Not Specified...

Edit Import Values from JSON

main.py Python

```python
from openai import OpenAI
client = OpenAI(api_key="{{SECRET_KEY}}")

response = client.chat.completions.create(
  model="gpt-3.5-turbo-0125",
  messages=[
    {"role": "system", "content": "You are a helpful assistant designed t
    {"role": "user", "content": "Write a tagline about artificial intelli
  ],
  temperature = 0.9
)
print(response.choices[0].message.content)
```

Ask

Completions endpoint sample run

Let's now see some details of the above code:

- **Line 1:** We import the `OpenAI` library.
- **Line 2:** We create an OpenAI client instance using the provided API key.
- **Line 4:** We initiate a chat completion request.
- **Line 5:** We specify the model to be used for generating responses.
- **Lines 6–9:** We provide a sequence of messages, including system and user messages.
- **Line 10:** We set the temperature parameter for response generation to `0.9`.
- **Line 12:** We print the content of the first choice of response from the generated completions.

Try out different prompts and examine the varying responses. Here are some examples:

```
"Suggest one name for a cat."
```

```
"Suggest two names for a cat."
```

```
"Suggest names for a cat."
```

In the first example, the API will return only one name. In the second example, it will return two names. For the third example, it will return multiple names. We'll see that the model is great at understanding the context of the sentences, and the

Ask